

Minesweeper Agent Using Double Deep Q-Network(DDQN) with CNN-Based State Representation

Yixiang Hua
Dept. of Electrical and Computer
Engineering
McGill University
Montreal, Canada
yixiang.hua@mail.mcgill.ca

Haorui Bao
Dept. of Electrical and Computer
Engineering
McGill University
Montreal, Canada
haorui.bao@mail.mcgill.ca

Abstract—This paper presents a reinforcement learning-based Minesweeper agent using a Double Deep Q-Network (DDQN) architecture combined with a convolutional neural network (CNN) state encoder. Minesweeper poses unique challenges for reinforcement learning because of its sparse rewards, mixed necessity of logic inference and guessing, and extremely large action space induced by hidden cells. To address these challenges, we propose (1) an 11-channel state encoding that converts the Minesweeper board into an image-like tensor, (2) a three-layer CNN to capture spatial constraints required for logical inference, (3) a DDQN structure to reduce overestimation bias during Q-learning, and (4) reward shaping that encourages progress while discouraging blind guessing. Experimental results show that the proposed method can learn meaningful local patterns and achieve non-trivial performance on 10×10 Minesweeper boards.

Keywords—component, formatting, style, styling, insert (key words)

I. INTRODUCTION & RELATED WORK

A. Problem Motivation

Minesweeper is a nontrivial environment for reinforcement learning because:

It is partially observable: most tiles are initially hidden.

Rewards are sparse, with success only at the end of the game.

The agent must perform logical inference, not just reward maximization.

Many states involve unavoidable guessing, making long-term predictions difficult.

Unlike typical RL benchmark environments (Atari, CartPole), Minesweeper is non-competitive and the purpose is not to maximize score but to maximize logical progress while minimizing blind guesses. This motivates a combination of deep learning and structured constraints.

B. Objectives

The goal of this project is to design an RL agent that:

1. Learns to uncover tiles through logical inference,
2. Minimizes random guessing,
3. Uses CNNs to interpret local patterns such as “1-2-1” or “2-3-2,”

4. Applies DDQN to stabilize learning and reduce Q-value overestimation.

C. Course Concepts

This project extends course concepts including:

1. Q-learning
2. Bellman updates
3. Value function approximation
4. Deep Q-Networks (DQN)
5. Double DQN to reduce overestimation
6. Experience replay

D. Related Work

[1] Lee, S. D. (2024). *Minesweeper AI with Reinforcement Learning*. Retrieved from

<https://sdlee94.github.io/Minesweeper-AI-Reinforcement-Learning/>

[2] Cai, C., Li, W., & Zhou, Y. (2025). *A Deep Reinforcement Learning Approach for Solving Minesweeper*. *Applied Sciences*, **15**(5), 2490.

<https://www.mdpi.com/2076-3417/15/5/2490>

II. METHOD

A. Environment Formulation (MDP)

The Minesweeper environment is formulated as an MDP:

State :

A 10×10 grid encoded into 11 channels (closed mask, one-hot digits, frontier mask).

Action :

The agent chooses one of 100 tiles.

Transition :

The environment reveals the tile according to the game's deterministic rules.

Reward :

Reward shaping encourages progress and penalizes blind guesses.

Discount Factor :

$\gamma = 0.01$ to stabilize value estimation and to avoid Q-value explosion.

B. State Encoding (11 Channels)

The raw Minesweeper board is converted into an 11-channel tensor:

1. Channel 1: Closed mask (-2)
2. Channels 2–10: One-hot encoding of digits 0–8
3. Channel 11: Frontier mask (tiles adjacent to opened cells)

This transforms the board into an image-like representation suitable for CNNs.

C. Neural Network Architecture

We use a 3-layer CNN:

Input: $10 \times 10 \times 11$
 Conv1: 64 filters, 3×3 , padding=1
 Conv2: 64 filters, 3×3
 Conv3: 128 filters, 3×3
 Flatten $\rightarrow$ 12800 units
 FC1 $\rightarrow$ 512
 FC2 $\rightarrow$ 100 (Q-values for 100 actions)

Deepening the network increases the receptive field, enabling the model to capture multi-tile patterns.

D. Frontier-Based Action Mask

To reduce exploration of meaningless states, the agent is allowed to click only frontier tiles (unopened tiles adjacent to opened ones), except for the first move.

This significantly improves sample efficiency and reduces guess frequency.

E. Reward Shaping

We introduce a dense shaped reward:

+1 : win

-5 : lose

+1 : logical progress (newly revealed tile adjacent to previously open cells)

-0.5 : guess (opening a tile not adjacent to known information)

This encourages inference-oriented behavior.

F. Double DQN Update

Standard DQN uses:

$$y = r + \gamma * \max_a Q_{\text{online}}(s', a)$$

which causes overestimation.

DDQN uses:

$$a * = \operatorname{argmax}_a Q_{\text{online}}(s', a)$$

$$y = r + \gamma * Q_{\text{target}}(s', a *)$$

The online network selects the action; the target network evaluates it.

This separation stabilizes training.

G. Experience Replay

A replay buffer of 500k transitions is used. Batch size = 64. And the target network updates every 2000 steps.

H. Discard training records if too few tiles opened

If fewer than ten tiles were opened during another training session and that session failed, we would discard the training record because the model couldn't learn anything from it.

III. RESULTS

A. Evaluation Metrics

We evaluate the agent using:

1. Average opened cells
2. Progress ratio
3. Guess ratio
4. Win rate
5. Episode reward

B. Agent Performance

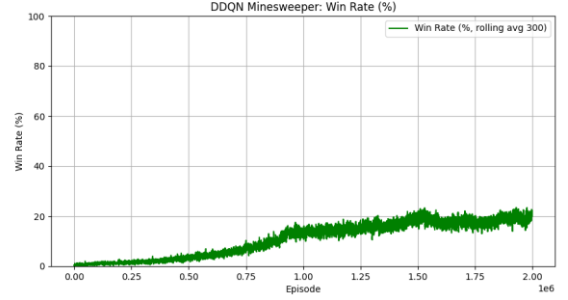


Fig. 1. Winrate before optimize

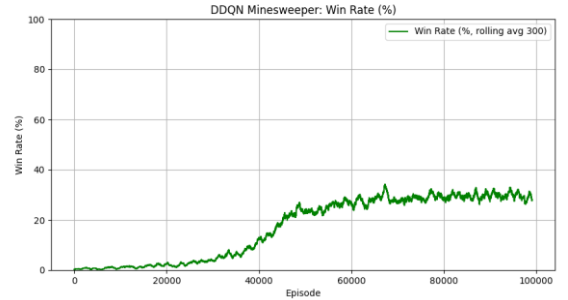


Fig. 2. Winrate after optimize

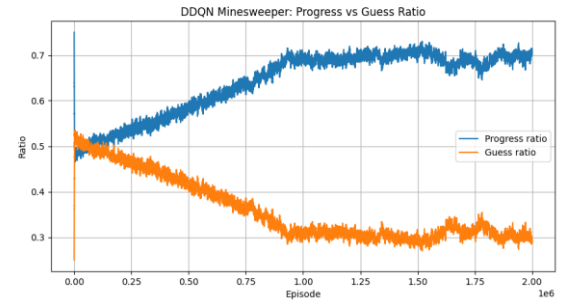


Fig. 3. Progress vs Guess Ratio before optimize

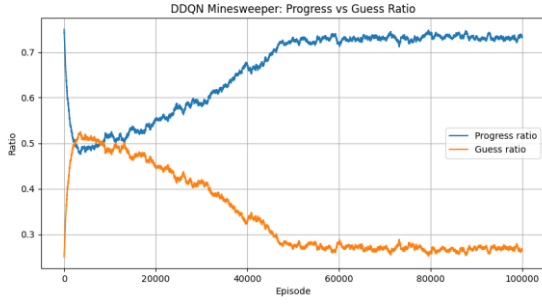


Fig. 4. Progress vs Guess Ratio after optimize

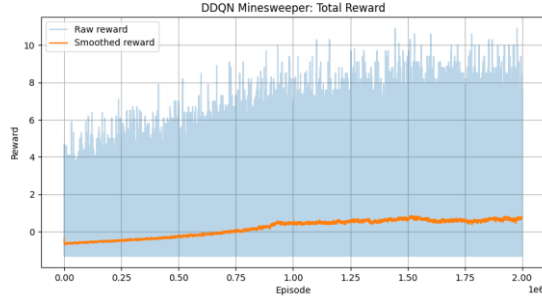


Fig. 5. Reward before optimize

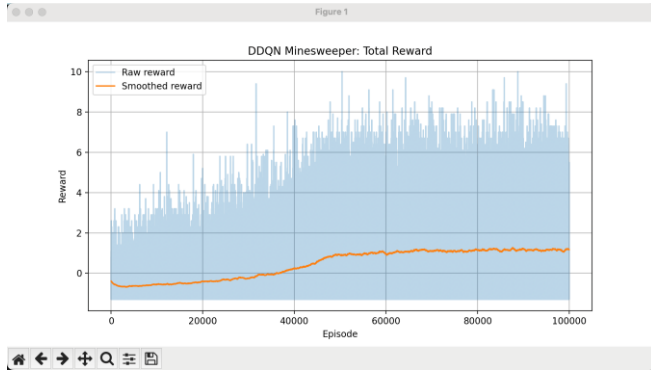


Fig. 6. Reward after optiize

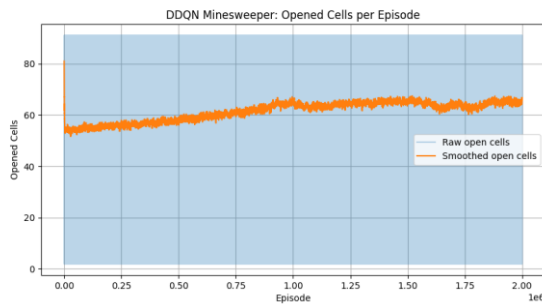


Fig. 7. Open cells before optimize

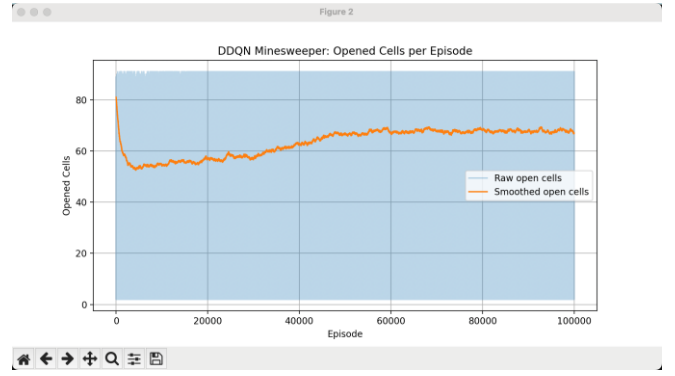


Fig. 8. Open cells after optimize

C. Summary

Progress ratio increased from $\sim 0.7 \rightarrow \sim 0.74$

Guess ratio decreased from $\sim 0.0.3 \rightarrow \sim 0.26$

Opened cells increased from $\sim 63 \rightarrow \sim 67$

Win rate increased from 25% $\rightarrow \sim 30\%$

IV. DISCUSSION

A. Successes

CNN successfully learns spatial patterns such as 1-2-1 structures.

DDQN reduces instability and prevents Q-value explosion.

Frontier masking significantly accelerates learning.

Reward shaping leads to more logical behavior.

B. Limitations

Minesweeper inherently includes unavoidable guessing.

Win rate remains low despite improved progress.

Sparse terminal rewards make long-term planning difficult.

C. Future Work

Add recurrent layers (LSTM/Transformer) to capture reasoning chains.

Combine probability-based solvers with RL.

Curriculum learning from $5 \times 5 \rightarrow 10 \times 10$ boards.

Imitation learning from human expert play.

V. CONCLUSION

This project demonstrates a DDQN-based agent capable of partially learning logical inference in Minesweeper. By combining CNN-based state encoding, reward shaping, frontier masking, and DDQN stability, the agent exhibits measurable improvements in inference-driven behavior. Although the win rate(30%) remains low due to the fundamental nature of the game, the results highlight the potential of deep RL applied to partially observable logical environments.

REFERENCES

[1] Lee, S. D. (2024). *Minesweeper AI with Reinforcement Learning*. Retrieved from

<https://sdlee94.github.io/Minesweeper-AI-Reinforcement-Learning/>

[2] Cai, C., Li, W., & Zhou, Y. (2025). *A Deep Reinforcement Learning Approach for Solving Minesweeper*. *Applied Sciences*, **15**(5), 2490.

<https://www.mdpi.com/2076-3417/15/5/2490>